EXP 1	**Develop a predictive text system using N-Gram and HMM (Hidden Markov Models) to predict the next word based on previous context.** **Dataset: Brown Corpus or Wikipedia dataset.**

Generative AI

AIM:

To develop a **predictive text system** that predicts the **next word based on previous context** using **N-Gram language models and Hidden Markov Models (HMM)**. The system is trained on a **text corpus such as the Brown Corpus or Wikipedia dataset** to learn word patterns and probabilities for generating accurate next-word predictions.

CODE:

EXP-1

1a. N-GRAM BASED PREDICTIVE TEXT SYSTEM (BIGRAM & TRIGRAM)

```
import nltk
import re
from nltk.corpus import brown
from nltk.tokenize import word_tokenize
from nltk.util import ngrams
from collections import Counter

# Downloads
nltk.download('brown')
nltk.download('punkt')
nltk.download('punkt_tab') # Added to resolve LookupError

# ------------------------------
# Load Dataset
# ------------------------------
sentences = brown.sents()
text = " ".join([" ".join(sent) for sent in sentences])

# ------------------------------
# Preprocessing
# ------------------------------
text = text.lower()
text = re.sub(r'[^a-z\s]', '', text)
tokens = word_tokenize(text)
tokens = [t for t in tokens if t.strip() != ""]

# ------------------------------
# Generate N-Grams
# ------------------------------
unigrams = list(ngrams(tokens, 1))
bigrams = list(ngrams(tokens, 2))
```

```python
# -------------------------------
# Frequency Count
# -------------------------------
unigram_freq = Counter(unigrams)
bigram_freq = Counter(bigrams)

vocab_size = len(set(tokens))

# -------------------------------
# Next Word Prediction (Bigram)
# -------------------------------
def predict_next_word_ngram(word):
    word = word.lower()

    # Context-specific candidates
    candidates = {
        w2: count
        for (w1, w2), count in bigram_freq.items()
        if w1 == word
    }

    if not candidates:
        return "No prediction available"

    # Laplace smoothed probabilities
    probs = {
        w: (count + 1) / (unigram_freq[(word,)] + vocab_size)
        for w, count in candidates.items()
    }

    return max(probs, key=probs.get)

# -------------------------------
# USER INPUT
# -------------------------------
user_word = input("Enter a word (N-Gram): ")
print("Predicted next word:", predict_next_word_ngram(user_word))
```

Output :

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Enter a word (N-Gram): we
Predicted next word: have
```

1b. HIDDEN MARKOV MODEL (HMM) BASED PREDICTIVE TEXT SYSTEM

```python
import nltk
import random
from nltk.corpus import brown
from nltk.tag import hmm

# Downloads
nltk.download('brown')
nltk.download('universal_tagset')

# -------------------------------
# Load POS Tagged Dataset
# -------------------------------
tagged_sentences = brown.tagged_sents(tagset='universal')

# -------------------------------
# Train HMM Model
# -------------------------------
trainer = hmm.HiddenMarkovModelTrainer()
hmm_model = trainer.train(list(tagged_sentences[:5000])) # Converted to list to prevent 'read of closed file' error

# -------------------------------
# Next Word Prediction using HMM
# -------------------------------
def predict_next_word_hmm(sentence):
    words = sentence.lower().split()
    tagged = hmm_model.tag(words)

    last_tag = tagged[-1][1]

    # Predict next POS tag
    next_tag = max(
        hmm_model._states,
        key=lambda tag: hmm_model._transitions[last_tag].prob(tag)
    )

    # Get word probabilities for predicted POS
    word_probs = {
        word: hmm_model._outputs[next_tag].prob(word)
        for word in hmm_model._outputs[next_tag].samples()
    }

    # Choose from top 5 probable words
    top_words = sorted(
        word_probs.items(),
        key=lambda x: x[1],
        reverse=True
    )[:5]

    return random.choice(top_words)[0]

# -------------------------------
# USER INPUT
# -------------------------------
user_sentence = input("Enter a sentence (HMM): ")
print("Predicted next word:", predict_next_word_hmm(user_sentence))
```

Output :

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.
Enter a sentence (HMM): they were playing
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:335: RuntimeWarning: overflow encountered in cast
  O[i, k] = self._output_logprob(si, self._symbols[k])
/usr/local/lib/python3.12/dist-packages/nltk/tag/hmm.py:333: RuntimeWarning: overflow encountered in cast
  X[i, j] = self._transitions[si].logprob(self._states[j])
Predicted next word: will
```

Result :

The predictive text system was successfully implemented using N-Gram and Hidden Markov Model techniques. The trained model was able to analyze the context of previous words and predict the most probable next word based on learned probability distributions from the dataset.

The N-Gram model captured local word dependencies effectively, while the HMM model helped model hidden linguistic structures and sequence probabilities. The system demonstrated the capability to generate context-aware next word predictions, showing how statistical language models can be used in applications such as autocomplete, search engines, and smart keyboards.